# Assignment — Python Concurrency & Async Execution

---

## □ Task 1 — Pure Synchronous Execution

> **Objective:** Understand execution blocking in synchronous programs.

**Requirements:**

- Create a function called: `run_sync_tasks()`
- Execute **8 tasks** using: `time.sleep()`
- Each task must print: `Start → delay → Finish`
- Calculate the **total execution time**

**Note :**

- Each task starts only after the previous one finishes
- Execution is **fully sequential**

---

## □ Task 2 — Async Sequential *(Without gather)*

> **Objective:** Understand that async does NOT automatically mean parallel.

**Requirements:**

- Use **async functions**
- Create an async task function
- Call tasks like this: `await task()` inside a loop

**Note :**

- Execution remains **sequential**
- Total execution time is similar to synchronous execution

---

## □ Task 3 — Async Concurrent Using `gather()`

> **Objective:** Understand real concurrency using asyncio.

**Requirements:**

- Create a **list of tasks**
- Use: `await asyncio.gather(...)` to execute them

**Note :**

- All tasks start **almost at the same time**
- Execution time becomes **significantly shorter**

---

# □ Task 4 — Blocking CPU Inside Event Loop

> **Objective:** Understand why CPU-bound operations break async workflows.

**Requirements:**

- Create a function called: `heavy_cpu_function()` that contains a large loop or heavy computation
- Run it **directly** inside an async main function

**Note :**

- Other async tasks **stop running**
- Event loop becomes **blocked**

---

# □ Task 5 — Yielding Inside Single Thread

> **Objective:** Understand cooperative multitasking.

**Requirements:**

- Inside a loop use: `await asyncio.sleep(0)`

**Note :**

- Event loop can **switch between tasks**
- Improved **responsiveness**

---

# □ Task 6 — Using `asyncio.to_thread()`

> **Objective:** Solve blocking issues using threads.

**Requirements:**

- Run the heavy CPU function using: `await asyncio.to_thread(...)`

**Note :**

- Light async tasks continue running **smoothly**
- Event loop remains **responsive**

---

# □ Task 7 — Race Condition Simulation

> **Objective:** Understand thread synchronization and race conditions.

**Requirements:**

- Create a shared variable: `shared_counter = 0`
- Run **multiple threads** modifying it
- Execute twice:
  - **Without Lock**
  - Using: `threading.Lock()`

**Note :**

- Different results between both cases
- **Race condition** problems without synchronization

---

# □ Task 8 — Async + Thread Synchronization Hybrid

> **Objective:** Combine async workflows with threads and synchronization.

**Requirements:**

- Async tasks should **request CPU work**
- CPU work must run **inside a thread**
- Thread modifies a **shared variable** protected by Lock

**Note :**

- Async orchestration with **thread safety**
- Parallel execution **without data corruption**

---

# Quick Reference Table

| # | Task | Key Concept |
|---|---|---|
| | Pure Synchronous Execution | `time.sleep()` · Sequential blocking |
| | Async Sequential | `await` in loop · Still sequential |
| | Async Concurrent | `asyncio.gather()` · True concurrency |
| | Blocking CPU | Heavy computation · Event loop blocked |
| | Yielding | `asyncio.sleep(0)` · Cooperative multitasking |
| | `asyncio.to_thread()` | Offload CPU work · Non-blocking |
| | Race Condition | `threading.Lock()` · Thread safety |
| | Async + Thread Hybrid | Combined workflow · Safe parallelism |